

APPLIED MATH FOR COMPUTER SCIENCE

CPCS 212

Lab 8: Interpolation and curve fitting

Interpolation and curve fitting

- Interpolating means that the function passes through all the given points with no regard to the points between them.
- Approximating means that the function is close to all the given points and probably close to nearby points as well.

Interpolation and curve fitting

- Polynomials are commonly used to interpolate data. There are several methods for defining, constructing, and evaluating the polynomial of a given degree that passes through data values.
- One common way is to solve for the coefficients of the polynomial by setting up a linear system called *Vandermonde's equation*.

Open the diary file
Lab8_yourname.txt

Vandermonde's Equation

- For the linear polynomial $p(x)=c_1x+c_2$ that passes through the data points (x_1,y_1) and (x_2,y_2) . We simply have to solve a set of linear equations for c_1 and c_2 .

$$c_1x_1 + c_2 = y_1$$

$$c_1x_2 + c_2 = y_2$$



$$\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

Vandermonde's Equation

- For the quadratic polynomial $p(x)=c_1x^2+c_2x + c_3$ that passes through the data points (x_1,y_1) , (x_2,y_2) and (x_3,y_3) . We simply have to solve a set of quadratic equations for c_1,c_2 .and c_3 .

$$c_1x_1^2+c_2x_1 + c_3 = y_1$$

$$c_1x_2^2+c_2x_2 + c_3 = y_2$$

$$c_1x_3^2+c_2x_3 + c_3 = y_3$$



$$\begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

Vandermonde's Equation

- It is characterized by the fact that for each row (sometimes column) of the coefficient matrix, the successive entries are generated by a decreasing (sometimes increasing) set of powers of a set of variables.
- It is possible to define a linear system that should be satisfied by the (unknown) polynomial coefficients. If we can solve the system, and solve it accurately, then that is one way to determine the interpolating polynomial.

construct the Vandermonde equation using Matlab

- Setting up the coefficient matrix A.

```
for j = 1:n  
    for k = 1:n  
        A(j,k) = xdata(j)^(n-k) ;  
    end  
end
```

- Where variables *xdata* and *ydata* are represent the quantities x_k and y_k and we will assume them to have length n .
- Then solving the linear system using $c = A \setminus ydata'$

We use the names:

- *xdata* and *ydata* to refer to known data values,
- *xval* will be a arbitrary point at which we want to evaluate the interpolating polynomial,
- *pval* will be the value of the polynomial at that point

1-(a)

- Write a Matlab function m-file, `coef_vander.m` with signature that accepts a pair of vectors `xdata` and `ydata` of arbitrary but equal length, and returns the coefficient vector `c` of the polynomial that passes through that data. Be sure to complete the comments with question marks in them.

1- (a)

```
function c= coef_vander(xdata,ydata)
    % c = coef_vander ( xdata, ydata )
    % xdata= given data  x points
    % ydata= given data y points
    % c= coefficient of the polynomial
    % n = length xdata vector
    % 1- setting up the coefficient matrix
    A
n = length(xdata);
for j = 1:n
    for k = 1: n
        A(j,k) = xdata(j) ^ (n-k);
    end
end
% 2- solving the linear system
c = A\ydata';
```

1- (b)

Test your function by computing the coefficients of the polynomial through the following data points. (*This polynomial is $y=x^2$, so you can check your coefficient vector “by inspection.”*)

```
>> % 1-(b)
```

```
>> xdata= [ 0 1 2 ] ;
```

```
>> ydata= [ 0 1 4 ] ;
```

```
>> c = coef_vander ( xdata, ydata )
```

1- (b)

```
>> xdata = [ 0 1 2];
```

```
>> ydata = [ 0 1 4];
```

```
>> c= coef_vader(xdata,ydata)
```

```
c =
```

```
    1
```

```
    0
```

```
    0
```

Where polynomial is : $y(x) = x^2$

1- (c)

Test your function by computing the coefficients of the polynomial that passes through the following points

```
>> % 1-(c)
```

```
>> xdata= [ -3 -2 -1 0 1 2 3]
```

```
>> ydata= [412 31 4 7 28 247 1636]
```

```
>> c = coef_vander ( xdata, ydata )
```

1-(c)

```
>> xdata = [ -3 -2 -1 0 1 2 3]
```

```
>> ydata = [ 412 31 4 7 28 247 1636]
```

```
>> c= coef_vader(xdata,ydata)
```

```
c =
```

```
1.0000
```

```
2.0000
```

```
3.0000
```

```
4.0000
```

```
5.0000
```

```
6.0000
```

```
7.0000
```

Where polynomial is :

$$y(x) = x^6 + 2x^5 + 3x^4 + 4x^3 + 5x^2 + 6x + 7$$

MATLAB polynomials commands

MATLAB polynomials commands

- polynomial $\mathbf{a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0}$ represent **by vector** $[\mathbf{a_n, a_{n-1}, \dots, a_1, a_0}]$
- Finds the coefficients of the polynomial whose roots are given by the vector $\mathbf{r}$.

$$\mathbf{c=poly(r)}$$

- Finds the roots of the polynomial whose coefficients are given by the vector $\mathbf{c}$.

$$\mathbf{r=roots(c)}$$

MATLAB polynomials commands

- Evaluates a polynomial with coefficients given by the vector c at the values $xval(k)$ for $k=1,2,\dots,k$ for $k \geq 1$.

$$yval = \text{polyval}(c, xval)$$

- Finds the coefficients of the polynomial of degree n passing through the points $xdata(k), ydata(k)$, for $k=1,2,\dots,k$ where $k-1$ need not be the same as n .

$$c = \text{polyfit}(xdata, ydata, n)$$

- The Matlab built-in function *polyfit* finds the coefficients of a polynomial through a set of points. It works as Vandermonde's equation that used in previous slides.

1-(d)

Confirm using *polyval* that your polynomial passes through these data points.

```
>> % 1-(d)
```

```
>> ydatdnew = polyval(c,xdata)
```

1-(d)

(b)

```
>> ydatanew = polyval(c,xdata)
```

```
ydatanew =
```

```
0      1      4
```

(c)

```
>> ydatanew = polyval(c,xdata)
```

```
ydatanew =
```

```
1.0e+003 *
```

```
0.4120    0.0310    0.0040    0.0070    0.0280    0.2470  
1.6360
```

1-(e)

Double-check your work by comparing with results from the Matlab *polyfit* function.

1- (e)

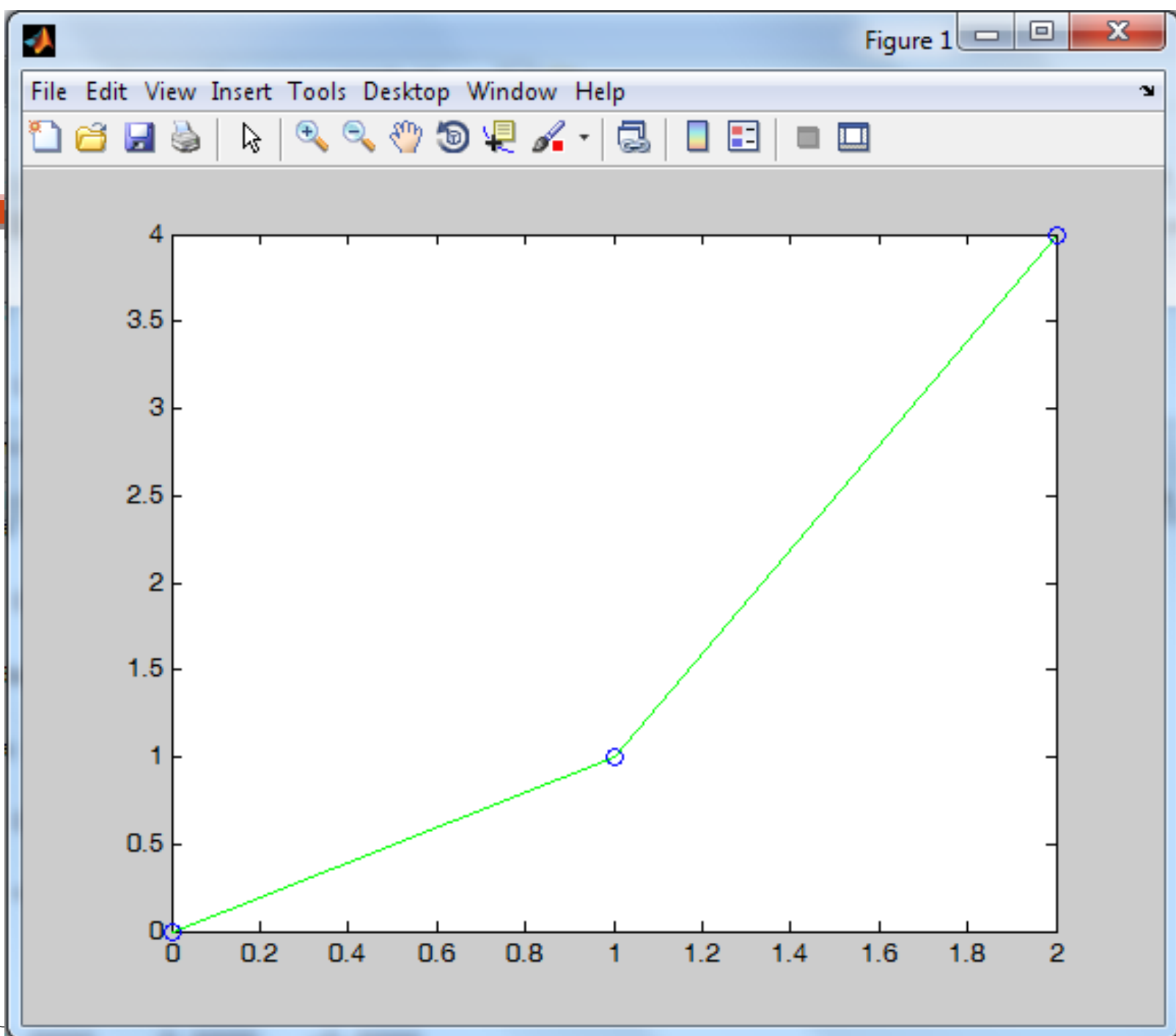
```
>> % 1-(e)
>> xdata = [ 0 1 2 ];
>> ydata = [ 0 1 4 ];
>> n=length(xdata) -1

>> c = polyfit(xdata,ydata,n)
c =
    1.0000    0.0000   -0.0000

>> y = polyval(c,xdata)
y =
   -0.0000    1.0000    4.0000

>> plot(xdata,ydata,'o',xdata,y,'g-');
```

(b)



1-(e)

```
>> xdata = [ -3 -2 -1 0 1 2 3]
```

```
>> ydata = [ 412 31 4 7 28 247 1636] (c)
```

```
>> c = polyfit(xdata,ydata,6)
```

```
c =
```

```
1.0000 2.0000 3.0000 4.0000 5.0000 6.0000 7.0000
```

```
>> yval = polyval(c,xdata)
```

```
>> yval =
```

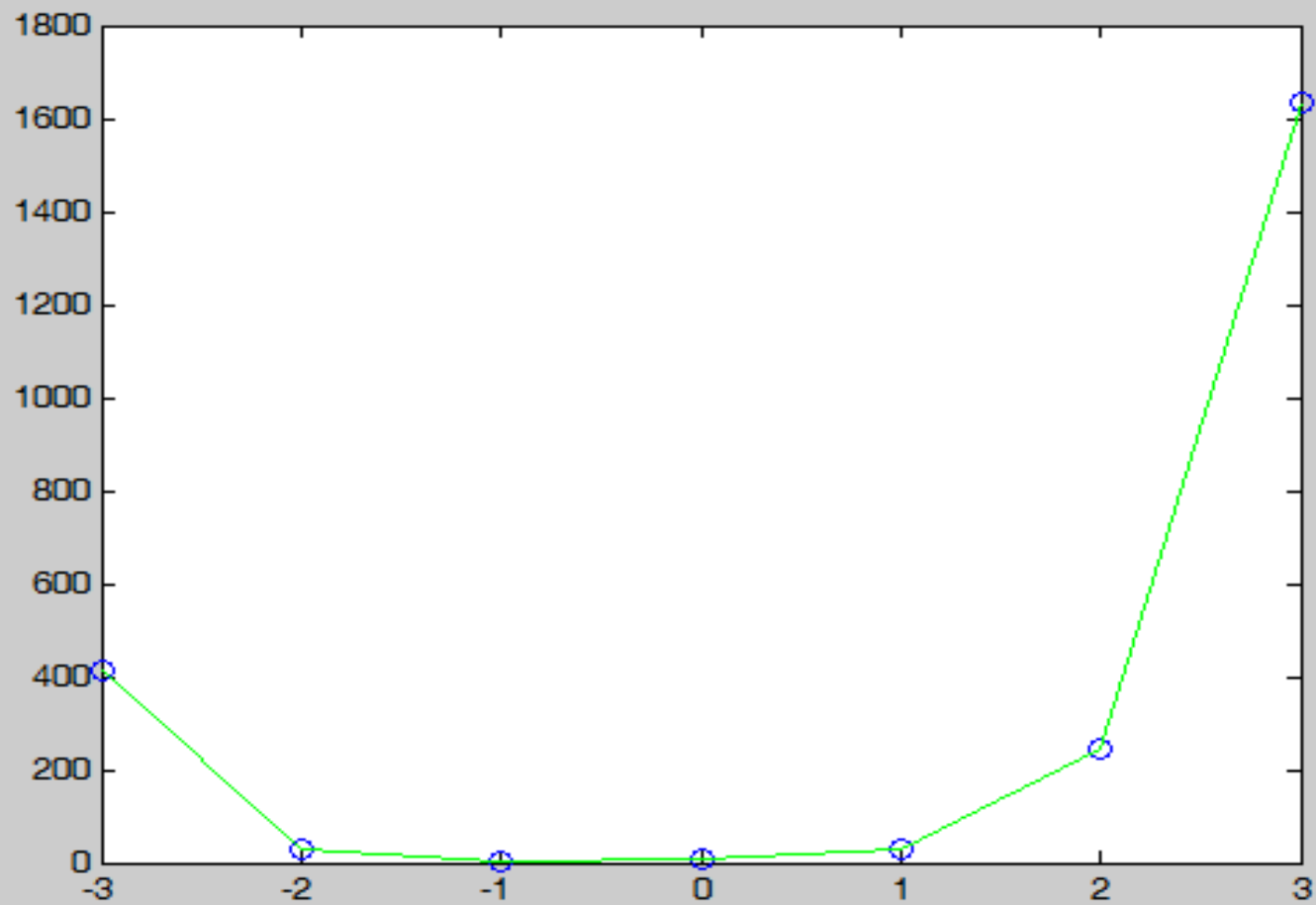
```
1.0e+003 *
```

```
0.4120 0.0310 0.0040 0.0070 0.0280 0.2470 1.6360
```

```
>> plot(xdata,ydata,'o',xdata,yval,'g-');
```


Figure 1

File Edit View Insert Tools Desktop Window Help



1-(e) (d)

```
>> xdata = [ -3 -2 -1 0 1 2 3]
>> ydata = [ 412 31 4 7 28 247 1636]
% polynomial is y=x2
c = polyfit(xdata,ydata,2)

c =

    120.4286    147.4286   -143.8571

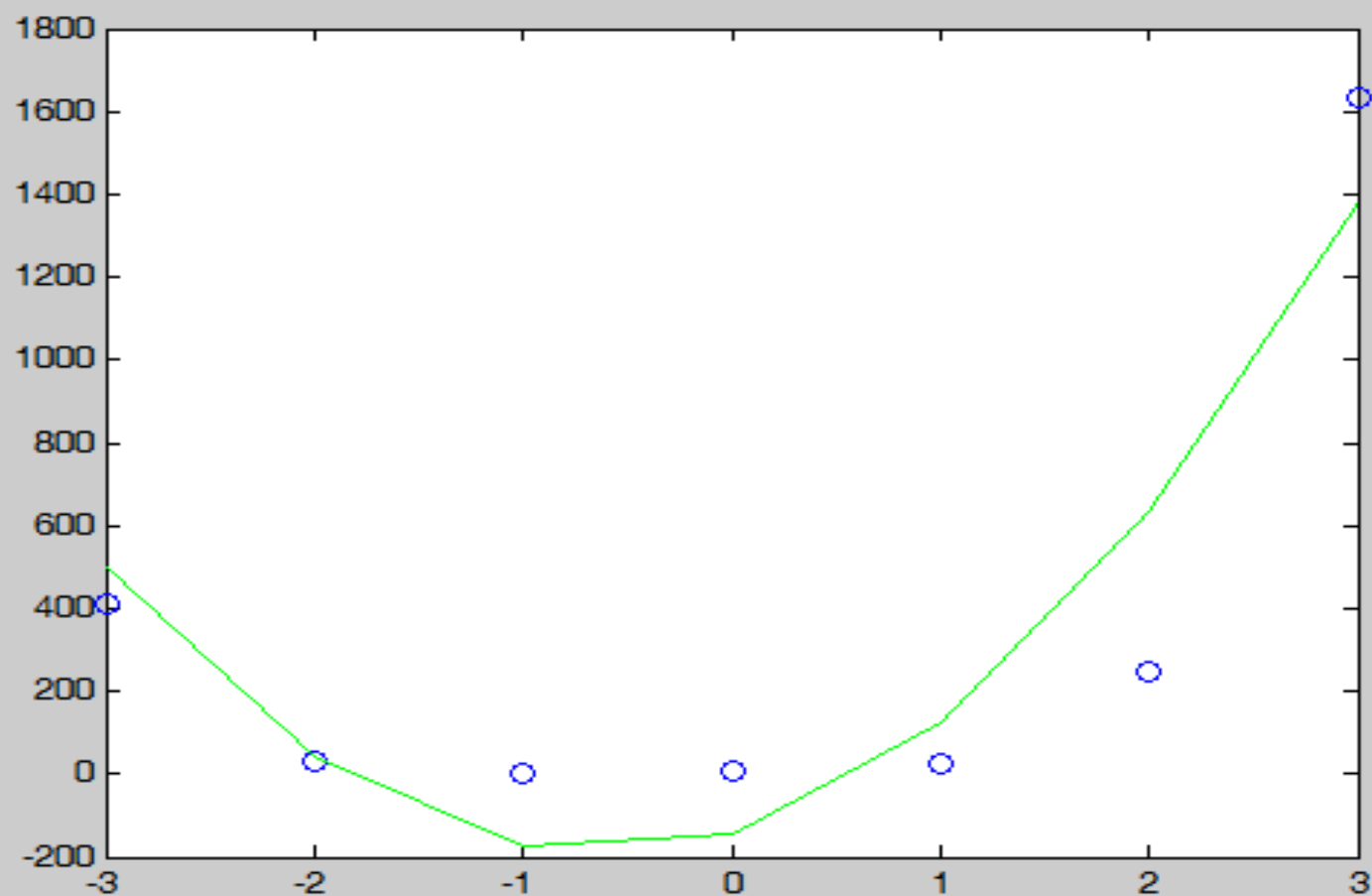
>> yval = polyval(c,xdata)

yval =
    1.0e+003 *
    0.4977    0.0430   -0.1709   -0.1439    0.1240    0.6327
    1.3823

plot(xdata,ydata,'o',xdata,yval,'g-');
```

Figure 1

File Edit View Insert Tools Desktop Window Help



Note

- When there are more data values than the minimum, the *polyfit* function returns the coefficients of a polynomial that “best fits” the given values in the sense of least squares.
- This polynomial approximates, but does not necessarily interpolate, the data.
- In this lab, you will be writing m-files with functions similar to *polyfit* but that generate polynomials of the precise degree determined by the number of data points. ($n = \text{length}(\text{xdata}) - 1$).

Question

Find the fourth-degree polynomial that fits the following five data points?

$$X = [-2, -1, 0, 1, 2]$$

$$Y = [3, 5, 1, 4, 10]$$

Solution

```
c=polyfit(X,Y,4)
```

```
c =
```

```
-0.7083  0.7500  4.2083 -1.2500  1.0000
```

The polynomial is

$$-0.7083x^4 + 0.7500x^3 + 4.2083x^2 - 1.2500x + 1.0000$$

interp1 Matlab Built-in Function

$Y_q = \text{interp1}(X_{\text{data}}, Y_{\text{data}}, x_q, \text{method})$

- *interp1* Matlab Built-in Function perform 1-D interpolation with one of the following 4 methods:
 - *nearest-neighbor* which uses piecewise-constant function,
 - *linear interpolation* that uses piecewise-linear polynomials,
 - *cubic interpolation* uses piecewise-cubic polynomials
Interpolant and $f'(x)$ are continuous
 - *spline interpolation* uses cubic splines.
 - The default method is 'linear'

2-(a)

Use the help command to learn how to use interp1.

```
>> % 2 -(a)  
>> help interp1
```


2-(b)

Use *interp1* to estimate the value of y at $x=2.5$ given the following:

```
>> % 2-(b) interp1
```

```
>> xdata= [ -3 -2 -1 0 1 2 3] ;
```

```
>> ydata= [412 31 4 7 28 247 1636];
```

```
>> yval = interp1(xdata,ydata,2.5)
```

```
% x = [ -2.5: .5:2.5];
```

```
% yval = interp1(xdata,ydata,x);
```

2-(b)

interp1 Matlab Built-in Function perform 1-D interpolation

```
>> xdata = [ -3 -2 -1 0 1 2 3]
```

```
>> ydata = [412 31 4 7 28 247 1636]
```

```
ydata =
```

412	31	4	7	28	247	1636
-----	----	---	---	----	-----	------

```
>> interp1(xdata,ydata,2.5)
```

```
ans =
```

```
941.5000
```

2-(b)

spline interpolation Matlab Built-in function uses Cubic spline data interpolation.

```
>> % 2-(b) spline  
>> xdata= [ -3 -2 -1 0 1 2 3] ;  
>> ydata= [412 31 4 7 28 247 1636];  
>> yval = spline(xdata,ydata,2.5)
```

```
%x = [ -2.5: .5:2.5];  
%yval = spline(xdata,ydata,2.5);
```

Example

```
xdata = 0:pi/4:2*pi;  
ydata = sin(xdata);  
xq = 0:pi/16:2*pi;  
subplot(1,2,1)  
yq1 = interp1(xdata,ydata,xq);  
plot(xdata,ydata,'o',xq,yq1,'-.');  
xlim([0 2*pi]);  
title('(Default) Linear Interpolation');  
subplot(1,2,2)  
yq2 = interp1(xdata,ydata,xq,'spline');  
plot(xdata,ydata,'o',xq,yq2,'-.');  
xlim([0 2*pi]);  
title('Spline Interpolation');
```

plot

